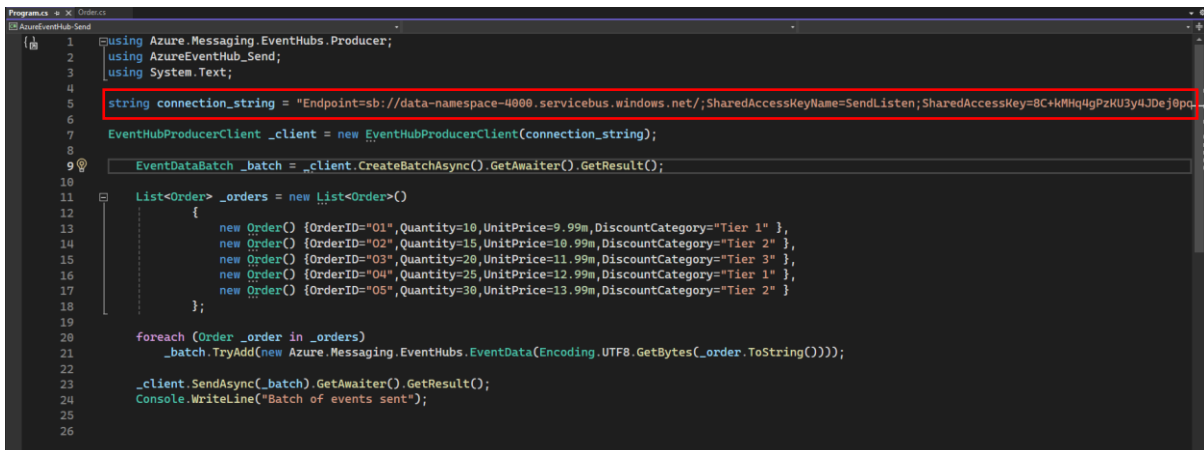


Sending Events

In this section, we're configuring a .NET program to send events to an Azure Event Hub. The end goal is to demonstrate the ability to programmatically send data to the Event Hub, enabling real-time ingestion of events for subsequent processing and analysis. This process forms a crucial part of building scalable and efficient data pipelines for various use cases, such as IoT data ingestion, log streaming, and real-time analytics, using Azure Event Hubs.

1. So, in the prior lab we had created an event hub namespace, and then we create something known as an event hub within that namespace. What's the next step? The next step is being able to send events onto the event hub, and also consume events from the event hub itself.
2. Now we are going to use a dot NET program to send these events.
3. You can get this program from GitHub in a zip format. Unzip it and then open it in Visual Studio 2022.
4. This is how the programming would look. Here you can see that we have also created a dataset in it.
5. To make use of this program we need to change the connection string.



```
1 using Azure.Messaging.EventHubs.Producer;
2 using AzureEventHub_Send;
3 using System.Text;
4
5 string connection_string = "Endpoint=sb://data-namespace-4000.servicebus.windows.net/;SharedAccessKeyName=SendListen;SharedAccessKey=BC+kMHq4gPzKU3y4JDej8pq...";
6
7 EventHubProducerClient _client = new EventHubProducerClient(connection_string);
8
9 EventDataBatch _batch = _client.CreateBatchAsync().GetAwaiter().GetResult();
10
11 List<Order> _orders = new List<Order>()
12 {
13     new Order() { OrderID="01", Quantity=10, UnitPrice=9.99m, DiscountCategory="Tier 1" },
14     new Order() { OrderID="02", Quantity=15, UnitPrice=10.99m, DiscountCategory="Tier 2" },
15     new Order() { OrderID="03", Quantity=20, UnitPrice=11.99m, DiscountCategory="Tier 3" },
16     new Order() { OrderID="04", Quantity=25, UnitPrice=12.99m, DiscountCategory="Tier 1" },
17     new Order() { OrderID="05", Quantity=30, UnitPrice=13.99m, DiscountCategory="Tier 2" }
18 };
19
20 foreach (Order _order in _orders)
21     _batch.TryAdd(new Azure.Messaging.EventHubs.EventData(Encoding.UTF8.GetBytes(_order.ToString())));
22
23 _client.SendAsync(_batch).GetAwaiter().GetResult();
24 Console.WriteLine("Batch of events sent");
25
26
```

6. For that in our event hubs instance we need to expand the settings option from the left and click on shared access policies.

   Consumer group  Delete  Refresh  Give feedback

 Overview

 Access control (IAM)

 Diagnose and solve problems

▼ Settings

 Shared access policies

 Configuration

 Properties

 Locks

 You can start generating test data with the new Azure Event Hubs Data

^ Essentials

Resource group ([move](#)) : [demo-grp](#)

Location : North Europe

Subscription ([move](#)) : [Azure Pass - Sponsorship](#)

Subscription ID : 3541d15a-44aa-4f6e-a120-1b7a6d5925bf

Partition count : [2](#)

7. Then we need to add a policy.

 Add

Policy

no policies have been set up yet.

8. Here you have to give your policy a name then choose send and listen properties. Then click on create.

Add SAS Policy



Event Hubs

Policy name *



☐ Manage

☒ Send

☒ Listen

9. Once your policy is created then you need to click on it. After that you will get the primary key and the secondary key plus the connection string.
10. So, copy one of the connection strings and paste it in the program.

SAS Policy: ProgramPolicy



Save Discard Delete Regenerate Primary Key ...

☐ Manage

☒ Send

☒ Listen

Primary key

eotNMQ13Ez8luNIReDqr27EHxcK72vxqT+AEhH83/wA=



Secondary key

Pgvs3n2PmEW9A/e7GS5CjBJPQcP8OmD/7+AEhKLmAzU=



Connection string–primary key

Endpoint=sb://demonamespace120.servicebus.windows.net;/SharedAccessKeyName=P...



Connection string–secondary key

Endpoint=sb://demonamespace120.servicebus.windows.net;/SharedAccessKeyName=P...



SAS Policy ARM ID

/subscriptions/3541d15a-44aa-4f6e-a120-1b7a6d5925bf/resourceGroups/demo-grp/p...



11. Once you have pasted the key then save your program and run it.
12. You will get a pop off message right away saying hat batch of event has been sent.

13. Go back to your event hub then go to your overview section and scroll down.
14. Below you can see that we the event hub have received some requests.

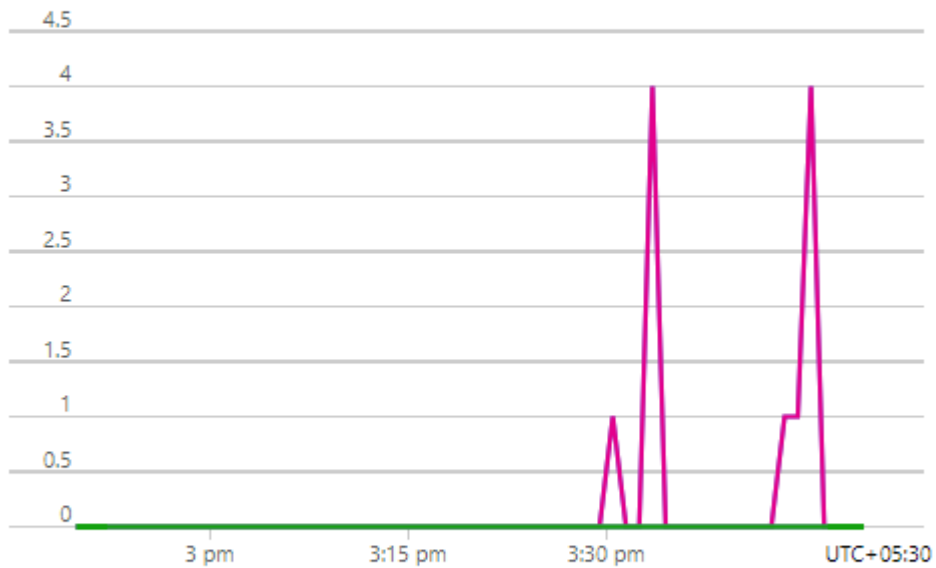
Event Hub Contents
1 CONSUMER GROUP

Event Hub status
ACTIVE

Cleanup policy
DELETE

Partition count
2

Requests



- 1/2
- Incoming Requests (Sum), demonamespace120 | 11
 - Successful Requests (Sum), demonamespace120 | 11
 - Server Errors. (Sum), demonamespace120 | 0